Activity 4: Party Roster Manager (Student Guide)

Printable copy: student_guide.pdf.

Start here

In Character Forge, you saved one character's information in a text file. In Party Roster, you will use a **database** to keep information about several characters, then make a window with buttons for working with that roster.

This guide is written for beginners. You are not expected to know database or window-programming terms before class. Start with the **Learning Primer** (four short readings with quick activities), then work through **Week 8**. Week 9 comes later in this same guide, after a clear stop page. Try one step at a time, and ask when a word or computer step is unclear.

What you will build

- **Week 8 (this session):** a text-menu program that adds characters, shows the roster, and deletes a character.
- **Week 9 (a later session):** a window with buttons for the same actions. Do not start it until Week 8 is finished and you have reached the stop page.

Both weeks use one file named `roster.db`. It is created by the program and remembers your roster after the program closes.

What you should be able to explain afterward

- How a database keeps related information in rows and columns.
- How Python asks the database to add, show, or delete a character.
- Why a `?` placeholder keeps a character's name separate from the SQL command.
- (After Week 9) How a button runs a function and how the window shows the updated roster.

Textbook connection

This is a **course in-class lab inspired by** *Murach's Python Programming*, Section 4; it is not a textbook exercise copied from the book. The assigned topics are:

- **Week 8 - Chapter 17, "How to work with a database":** database tables, SQL commands, and using Python with SQLite.
- **Week 9 - Chapter 18, "How to build a GUI program":** windows, widgets, layout, and responding to button clicks.

You will practice the same ideas using a guided RPG roster project.

Separate textbook homework is available for Chapter 17 Exercise 17-2 and Chapter 18 Exercise 18-2. Editable Markdown sources are in the same `homework` folder. Homework uses the textbook exercises and Canvas submission rules; it is not an extension of this in-class lab.

Before you code: Learning Primer

Read these four short sections **before** you start coding Week 8. Each one teaches one idea you will need, then gives you a quick activity so you can try it yourself. You do not need the project files for the activities. Use a scratch file in VS Code, or just paper. The answers are at the end of this primer.

- **Reading 1:** lists, tuples, and dictionaries side by side
- **Reading 2:** what a database is
- **Reading 3:** the four SQL commands
- **Reading 4:** how Python talks to the database (`execute`, `?`, and `commit`)

Reading 1 - Lists, tuples, and dictionaries side by side

You already know lists and dictionaries. A **tuple** is a third kind of group. It looks like a list, but it uses parentheses instead of square brackets:

```
a_list = ["Mira", 24, 3, "Mage"]
a_tuple = ("Mira", 24, 3, "Mage")
a_dict = {"name": "Mira", "hp": 24}
```

Kind	Written with	Can you change it after making it?	Items found by
List	`[ ]`	Yes: add, remove, replace items	Position (0, 1, 2...)
Tuple	`( )`	No: it is fixed once made	Position (0, 1, 2...)
Dictionary	`{ }`	Yes	A key name, like `"name"`

Why does this matter? When Python hands a few values to the database, the values are a **fixed, ordered set**. They are handed over once and used. Nothing needs to be added or removed, and the order is what matters. A tuple fits that job. A dictionary has names rather than positions, and a list is meant to change.

Where would you use a tuple in real life? Whenever a few values belong together as one fixed bundle:

- **A map location:** `(37.98, -121.31)` is latitude and longitude. They only make sense as a pair, and you would not add a third item or swap one out.
- **A color:** `(255, 103, 29)` is the red, green, and blue amounts. That one is Pacific orange.
- **A date:** `(2026, 10, 22)` is year, month, and day.
- **Two answers from one function:** `return low, high` hands back a tuple.

Because a tuple cannot be changed, no other part of the program can quietly alter your location or color. That is the safety you get compared with a list.

Special rule: a tuple with only one value needs a trailing comma, like `(7,)`. Without the comma, `(7)` is just the number 7 inside parentheses. You will rarely choose to make a one-item tuple. It happens when a tool insists on a tuple and you only have one value to give it. In this lab, deleting a character uses one `?`, so `execute` needs a tuple holding that one ID, such as `(2,)`.

Try it: make the three groups above in a scratch file and print each one. Then try `a_list[0] = "Bram"` and `a_tuple[0] = "Bram"`. One works and one gives an error. Which is which? Finally, print `type((7))` and `type((7,))`.

Reading 2 - What a database is

You may think of a database table as a simple spreadsheet:

id	name	hp	level	char_class
1	Mira	24	3	Mage
2	Bram	40	2	Warrior

- A **database** is a place where a program keeps organized information. Here it is the file `roster.db`.
- A **table** is one organized collection of information. Our table is called `characters`.
- A **row** is one character. A **column** is one kind of information, such as `hp`.
- The **id** is a number the database assigns to each character. It lets us delete the right row, even if two characters have the same name.
- **SQLite** is the database technology used here. It stores the database in one local file; no server or account is needed.
- **SQL** is the language used to ask a database to do something. Python sends SQL commands to the database for us.

An analogy helps, but a database is not literally a spreadsheet: the program uses clear commands to add, find, and remove records, and the database saves those changes in its file.

The big difference from the text file in Character Forge is that the file is **remembered and organized**. The program can ask for exactly the rows it wants, and the data is still there the next time the program runs.

Try it (paper): draw a table called `items` with the columns `id`, `name`, and `weight`. Fill in three rows. Which column would you use to point to exactly one row, and why?

Reading 3 - The four SQL commands

Python does not change the database directly. It sends a request written in **SQL**. You do not have to memorize SQL. Just learn to recognize what each command asks for:

SQL words	What they ask the database to do	Everyday example
`CREATE TABLE`	Set up the table and its columns	Drawing the empty form
`INSERT INTO`	Add a new row	Writing a new line on the form
`SELECT`	Get row(s) to look at	Reading lines from the form
`DELETE FROM`	Remove row(s)	Crossing a line off the form

A real command reads almost like a sentence:

```
SELECT name, hp FROM characters
```

This means: "Get the `name` and `hp` columns from the `characters` table."

Try it: which of the four commands matches each request? (a) "Show me everyone in the party." (b) "Remove the character with ID 2." (c) "Add a new Rogue named Finn." (d) "Make the empty roster table for the first time."

Reading 4 - How Python talks to the database

This is the part that is new to most students, so go slowly. Three ideas.

Idea 1: `conn.execute(...)`**.** `conn` is short for connection, the open link to the database. `execute` is an action `conn` can perform: "run this SQL command." You give it up to **two things**, separated by a comma:

- **The SQL command**, written as a string in quotes. This is always required.
- **The values** to fill in, written as a tuple (see Reading 1). This is **optional**. You need it only when the SQL contains `?` marks.

A command with no `?` needs only the first item:

```
conn.execute("SELECT name FROM characters")
```

A command with `?` marks needs both:

```
conn.execute("DELETE FROM characters WHERE id = ?", (2,))
```

The tuple has one item because there is one `?`. Notice the comma in `(2,)`. That is the one-item rule from Reading 1.

Idea 2: `?` **placeholders.** A `?` is a blank. Python fills the blanks, in order, with the items from the tuple:

```
conn.execute(
    "INSERT INTO characters (name, hp, level, char_class) VALUES (?, ?, ?, ?)",
    ("Mira", 24, 3, "Mage"),
)
```

Blank	Gets this value	Goes into column
1st ?	`"Mira"`	`name`
2nd ?	`24`	`hp`
3rd ?	`3`	`level`
4th ?	`"Mage"`	`char_class`

The number of `?` marks must match the number of values. If they do not, Python reports an error that mentions the number of bindings.

Why not just put the name straight into the command text? Imagine asking a helper to write a name on a form. The form's instructions should stay separate from the name being written on it. Placeholders keep the SQL instructions separate from what the user typed. That stops a name with quote marks or SQL-looking text from changing the command. This risk is called **SQL injection**. Do not build a command by putting a user's text inside an f-string. Always use `?` and send the values separately.

Idea 3: `conn.commit()`**.** After a change (`INSERT` or `DELETE`), the change is not permanent yet. `commit` means "keep this change in the database file." Without it, the change may be lost when the program closes. You commit after changes, not after a plain `SELECT`.

Try it: look at this command. (a) How many `?` marks are there? (b) Write the tuple that adds a Rogue named Finn with 18 HP at level 1. (c) What is wrong with `("Mira", 24, 3)`?

```
conn.execute(
    "INSERT INTO characters (name, hp, level, char_class) VALUES (?, ?, ?, ?)",
    ...
)
```

Primer answers

Check your answers after you try each activity.

- **Reading 1:** the list assignment works and the tuple assignment raises a `TypeError`, because a tuple cannot be changed. `type((7))` is `int` and `type((7,))` is `tuple`.
- **Reading 2:** `id` points to exactly one row, because every row gets a different number even if two names or weights match.
- **Reading 3:** (a) `SELECT`, (b) `DELETE FROM`, (c) `INSERT INTO`, (d) `CREATE TABLE`.
- **Reading 4:** (a) four. (b) `("Finn", 18, 1, "Rogue")`. (c) It has three values for four `?` marks, so Python reports an error about the number of bindings.

Open the project and run the Week 8 starter

These directions assume VS Code on Windows, as used in class.

1. Open the course project folder in VS Code. In the left **Explorer** panel, find and expand `activity_4_party_roster`, then expand its `code` folder.
2. Open `roster_db_starter.py`.
3. In VS Code's top menu, choose **Terminal > New Terminal**. A terminal is a place to type a command; it is not another file in your project.
4. Check that the terminal is working in the `activity_4_party_roster` folder. If it is at the course project folder above it, type `cd activity_4_party_roster` and press Enter. If it is already in `activity_4_party_roster`, do not run that command again.

5. Run the starter: `python code/roster_db_starter.py`.

The Week 8 starter prints the menu labels and exits until you complete its menu TODO. That is expected.

Important: always run the program from the same `activity_4_party_roster` folder. The program saves `roster.db` in the folder the terminal is currently using. Running it from a different folder can make a second, separate roster file seem to appear.

Week 8 - Make the database work

The idea before the code

Think of the program as a helper between you and the database:

```
You choose an action -> Python sends a request -> the database changes or returns rows
```

For example, when you choose "Add character," Python sends an `INSERT` command. When you choose "List roster," Python sends a `SELECT` command and then displays the rows it gets back.

The database work is divided into small functions. Each function has one responsibility:

Function	In everyday words
`init_db(conn)`	Make sure the roster table exists
`add_character(...)`	Save one character as a new row
`list_characters(conn)`	Get all the character rows
`delete_character(conn, character_id)`	Remove the row with that ID

`conn` is short for **connection**. It is the open link Python uses to talk to the database. You do not need to create a server or log in.

The four SQL commands

Reading 3 in the Learning Primer introduces these. Keep this reminder handy:

SQL words	What they ask the database to do
`CREATE TABLE`	Set up the table and its columns
`INSERT INTO`	Add a new row
`SELECT`	Get row(s) to look at
`DELETE FROM`	Remove row(s)

TODO 1 - Create the table once

The table needs a place for each character's ID, name, HP, level, and class. A simplified example of the SQL shape is:

```
CREATE TABLE IF NOT EXISTS characters (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    hp INTEGER NOT NULL,
    level INTEGER NOT NULL,
    char_class TEXT NOT NULL
)
```

Here, `TEXT` means words, `INTEGER` means a whole number, and `NOT NULL` means a value is required. `PRIMARY KEY AUTOINCREMENT` asks SQLite to assign a new ID for each row. `IF NOT EXISTS` means it should not try to make a duplicate table when you run the program again.

Your task: put a `conn.execute(...)` call in `init_db(conn)` with this table definition. On the next line, type `conn.commit()`. The word "commit" means "keep this change in the database file." (Without it, the change may be lost when

the program closes.)

Check your understanding: Which column gets an ID automatically? Which columns hold words?

TODO 2 - Add one character safely

The database needs the column names and values in the same order. This example adds a Mage:

```
conn.execute(
    "INSERT INTO characters (name, hp, level, char_class) VALUES (?, ?, ?, ?)",
    (name, hp, level, char_class),
)
conn.commit()
```

`conn.execute(...)` receives two things here: the SQL text and a tuple of four values, one for each `?`. If this is not clear yet, reread **Reading 4** in the Learning Primer before you type.

Your task: use this pattern in `add_character(...)`, then commit the change. The starter's `name`, `hp`, `level`, and `char_class` parameters already contain the values to save.

TODO 3 - Get and display the roster

To get the rows, run a `SELECT`, then ask for all returned rows:

```
cursor = conn.execute(
    "SELECT id, name, hp, level, char_class FROM characters"
)
rows = cursor.fetchall()
```

`fetchall()` means "give me all the rows this query found." Python receives a list of rows. Each row is a tuple of values in the same order as the `SELECT` list. For example:

```
(1, 'Mira', 24, 3, 'Mage')
```

Your task: make `list_characters(conn)` return the rows from `cursor.fetchall()`. The supplied `print_roster(rows)` function already knows how to display them.

Check your understanding: In the example row, which value is the ID? Which value is the class?

TODO 4 - Delete by ID, not by name

The database gave each character a unique ID. Use that ID to remove exactly one row:

```
cursor = conn.execute(
    "DELETE FROM characters WHERE id = ?",
    (character_id,),
)
conn.commit()
return cursor.rowcount > 0
```

`WHERE id = ?` tells SQLite which row to remove. The one-item tuple needs a comma after `character_id`: `(character_id,)`. The result is `True` if a row was removed and `False` if that ID was not in the roster.

Your task: use this pattern in `delete_character(...)`. The menu should only say "Deleted" when the function reports that an actual row was found.

TODO 5 - Build the menu loop

The menu lets someone repeat actions until choosing Quit. A **loop** repeats instructions; `while True` is a common way to keep showing a menu until a `break` ends the loop.

Plan the menu in plain language before typing:

1. Show the four choices: add, list, delete, quit.
2. Ask the person to choose one.

3. Use `if` / `elif` to perform the matching action.
4. If the choice is Quit, use `break` to leave the loop.
5. Close the database connection after leaving the loop.

For Add, ask for a name, class, HP, and level. Use `.strip()` to remove extra spaces around the text. Use `int(...)` to convert HP and level from typed text into whole numbers. Typed input starts as text, even when it looks like `24`.

Sometimes the person types `many` instead of `24`. `int("many")` cannot work; Python raises a `ValueError`. Put the conversion inside `try` / `except ValueError` so the program can show a helpful message and return to the menu instead of stopping. Also reject blank names/classes and HP or level below 1. A rejected entry should not be saved.

For Delete, check that the ID is a positive whole number too. An ID can be a number but still not exist, so report "not found" when `delete_character(...)` returns `False`.

Tip: catch `ValueError` around the `int(...)` conversion, not around the whole menu. That way, unexpected programming or database errors are not mistaken for ordinary typing mistakes.

Week 8: work in small steps

After each TODO:

1. Save the file with **Ctrl+S**.
2. Run `python code/roster_db_starter.py` again.
3. Try the behavior you just added.
4. If there is an error, read the last line of the error message. It often names the error type. Check the line number in your file and the spelling/indentation around it.

The menu itself is the final TODO. Until it is complete, you can also test the database functions with a small separate test or by completing the menu in the guided order.

Week 8: test the whole program

Try these checks and record what actually happened:

Test	What to do	What should happen
Add and list	Add Mira (24 HP, level 3, Mage), then list the roster	Mira appears with an ID and the details you entered
Delete	Delete Mira using the ID shown in the list	Mira is removed; listing again shows the roster is empty
Persistence	Add another character, choose Quit, then run the program again	The character is still there because it was saved in `roster.db`
Invalid number	Try entering `many` or `0` for HP/level	A helpful message appears, the program continues, and no invalid character is saved
Unknown ID	Try deleting a large ID that is not in the roster	The program says no character with that ID was found

If something does not match the expected result, write down what you typed and what the program actually did before changing code.

STOP: Week 8 ends here

Do not go past this page until Week 8 is finished. Week 9 is a separate session.

Before you stop, check that all of these are true:

- [ ] `init_db`, `add_character`, `list_characters`, and `delete_character` all work.
- [ ] The menu loop runs, and Quit ends the program.
- [ ] You ran every check in the Week 8 test table and wrote down what actually happened.
- [ ] Your character is still there after closing and re-running the program.
- [ ] Your work is saved (Ctrl+S).

If something is not done, ask for help or finish it before Week 9. Week 9 builds directly on this code, so unfinished Week 8 work will cause problems later.

Week 9 - Put a window in front of the database

Read before you code: Windows, buttons, and functions

Week 9 gives your program a window. A **GUI** (graphical user interface) is a window with things you can see and use. The window is built from small pieces called **widgets**:

- A `Label` shows words, such as "Name".
- An `Entry` is a box where the user types.
- A `Button` is something to click.

A program with a window does not run from top to bottom and stop. It **waits** for the user. When the user clicks a button, the window runs one of your functions. That function is called a **callback**, because the window "calls you back" when something happens.

You connect a button to a function with `command=`:

```
def say_hello():
    print("Hello!")

button = ttk.Button(root, text="Greet", command=say_hello)
```

Note that it is `say_hello` with **no parentheses**. You are telling the button *which* function to run later. If you wrote `say_hello()`, Python would run it immediately, once, before the window even appears.

Your button callbacks will do the same job as the Week 8 menu choices: read the text from the boxes, call a database function, then refresh the list on screen.

Try it: in the example above, what does the user see after clicking the button? What would happen if the code said `command=say_hello()` instead?

Answer: the user sees `Hello!` printed in the terminal. With `command=say_hello()`, the function would run once when the button is created, and clicking the button would then do nothing.

Open and run the Week 9 starter

1. In VS Code's Explorer panel, open `activity_4_party_roster/code/roster_gui_starter.py`.
2. Open a terminal (**Terminal > New Terminal**) and make sure it is in the `activity_4_party_roster` folder, as in Week 8.
3. Run `python code/roster_gui_starter.py`.

The Week 9 starter opens a mostly empty window until you add the form fields. That is expected. Use the same folder as Week 8 so the program finds your existing `roster.db`.

The idea before the code

A **GUI** (graphical user interface) is a window with things you can see and use, such as buttons and text boxes. Instead of typing `1` or `2` into a menu, you click a button.

The database functions from Week 8 are already included at the top of `roster_gui_starter.py`. You are building the window part and connecting it to those functions. The trip through the program is:

```
Type in a box -> click a button -> callback checks the text -> database changes
-> roster refreshes in the window
```

Meet the window pieces

The reading at the start of Week 9 explains these ideas with an example. Use this table as a reminder:

Word	What it means in this project
Window / root	The main application window

Word	What it means in this project
Widget	A visible piece of the window
`Label`	A short instruction or field name, such as "Name"
`Entry`	A box where the user types
`Button`	A clickable action, such as "Add Character"
`Treeview`	A table-like list that shows the roster
`Frame`	A container that groups related widgets, like the four input fields
Callback	A function Tkinter runs in response to an action, such as a button click

TODO 1 - Place labels and text boxes

The `RosterApp` class represents the roster window. It receives the main window (`root`) and the database connection (`conn`). The `form_frame` is already made for you; it is a smaller area inside the window for the input fields.

Use one row for each label and Entry:

```
tk.Label(form_frame, text="Name").grid(row=0, column=0, sticky="w")
self.name_entry = tk.Entry(form_frame)
self.name_entry.grid(row=0, column=1)
```

This creates a Name label in row 0, then a text box beside it in the same row. Repeat for HP, Level, and Class using rows 1, 2, and 3. Store each box as `self.name_entry`, `self.hp_entry`, `self.level_entry`, and `self.class_entry`. The word `self` lets the other methods in this window find the same boxes later.

The `.grid(row=..., column=...)` part places each widget in an invisible grid. Row 0 is the first horizontal line; column 0 is the first vertical position. `sticky="w"` means line the label up toward the left ("west") side of its cell.

Check: Can you point to the four labels and the four text boxes? Is each label beside the correct box?

TODO 2 - Make Add Character work

The Add button is already connected to `self.on_add_click`. This connection is called a **callback**: Tkinter waits, and runs the named function only when the user clicks the button.

The callback should do these things in order:

1. Read the boxes with `.get()`. This gives you text.
2. Use `.strip()` to remove spaces at the beginning/end of names and classes.
3. Check that name and class are not blank.
4. Convert HP and Level to integers with `int(...)`. Use `try` / `except ValueError` because the user might type words instead of numbers.
5. Check that HP and Level are at least 1.
6. If anything is invalid, show `messagebox.showerror(...)` and `return`. Do not clear the fields; the user can fix what they typed.
7. If everything is valid, call `add_character(self.conn, name, hp, level, char_class)`.
8. Clear the four boxes with `entry.delete(0, tk.END)`, then refresh the roster.

`return` exits this callback early. It is important after showing an error: it prevents invalid text from reaching the database.

The entries give you text, even for numbers. For example, `self.hp_entry.get()` might be `"24"`. `int("24")` converts that text into the number `24`; `int("many")` raises `ValueError`, which your callback should catch and explain politely.

TODO 3 - Show the latest roster

The `Treeview` is the table-like list on screen. The `refresh_list()` function should:

1. Remove the rows currently shown in the `Treeview`.
2. Call `list_characters(self.conn)` to get the latest database rows.
3. Insert each row into the `Treeview`.

Example of adding one row:

```
self.tree.insert("", tk.END, values=row)
```

`row` contains the five values in this order: ID, name, HP, level, class. The Treeview was set up with columns in that same order. Refreshing after an add or delete makes the window match what is saved in the database.

TODO 4 - Delete the selected character

The person must select a row in the Treeview before Delete can know which character to remove.

1. Ask for the selection with `self.tree.selection()`.
2. If there is no selection, show an information message and `return`.
3. Get the selected row's values with `self.tree.item(selected[0], "values")`.
4. The first value (`values[0]`) is the database ID. Convert it to an integer.
5. Call `delete_character(self.conn, character_id)`, then refresh the Treeview.

Use the ID, not the name. Two characters may have the same name, but their IDs are different.

How a button is connected

Look at this line, already in the starter:

```
add_button = tk.Button(form_frame, text="Add Character", command=self.on_add_click)
```

`text="Add Character"` is the words printed on the button. `command=self.on_add_click` tells Tkinter which function to run when it is clicked. Notice there are no parentheses after `on_add_click`: this gives Tkinter the function to run later instead of running it immediately. The Delete button is connected the same way.

At the end of the file, `root.mainloop()` keeps the window open and waits for clicks. Without this event loop, the window would appear and close immediately.

Week 9: test the whole window

Test	What to do	What should happen
Add	Enter Mira, 24, 3, Mage and click Add Character	Mira appears in the roster; the four boxes clear
Delete	Select Mira's row and click Delete Selected	Mira disappears from the roster
Empty field	Leave Name or Class blank and click Add	An error explains what is missing; no row is added
Invalid number	Enter `many` or `0` for HP/Level and click Add	An error appears; the typed values stay in the boxes; no row is added
No selection	Click Delete Selected without selecting a row	A message explains that you must select a character
Same database	Add a character in Week 8, then open the GUI from the same folder	The character appears in the window

Common computer and coding problems

What you see	What to check
`python` is not recognized	Ask the instructor to check which Python installation VS Code is using. Do not install a package from the internet on your own.
The roster is empty after switching from console to GUI	Make sure both runs started from the `activity_4_party_roster` folder. The database file is created in the terminal's current folder.
A button appears but nothing happens	Check that its `command=` names the correct method, that the method is indented inside `RosterApp`, and that the terminal does not show an error.
A red error appears in the terminal	Read the last line and note the line number. Check the spelling and indentation on that line, then ask for help if you are unsure.
The window will not start because Tkinter is missing	Tkinter normally comes with Python on class computers. Ask the instructor to check the Python installation; do not try `pip install tkinter`.

When asking for help, say which file you ran, what you clicked or typed, what you expected, and what happened instead. You do not need to know the name of the error to ask a clear question.

Using AI as a tutor

Useful questions:

- "Explain what one row and one column mean using this character table."
- "In my `INSERT`, explain how the four question marks match the four values. Do not rewrite my function."

- "What does this Python error mean in plain language? Here is the line number and what I typed."
- "My Add button does not respond. Help me check the `command=` line and callback connection."

Read any suggestion before using it. Be able to explain the line you keep. Never paste passwords, private information, or another student's work into an AI tool. Record meaningful AI help and how you checked it in your lab journal; say if you did not use AI.

Lab submission checklist

Party Roster is an **in-class lab**, so use the lab deliverables below. Textbook homework has a different submission standard; do not substitute the homework journal rules for these lab requirements.

Submit through Canvas

1. **Git repository link:** link to your Party Roster project or its folder, using the access settings the instructor approved.
2. **Your Python source files:** upload your completed `.py` files or a ZIP containing those files and any small, safe sample data. The upload must match the version in Git.
3. **Your lab journal:** write three things you learned in your own words, give one example of a skill used in your code, describe one problem you investigated/fixed (or a check you performed if there was no problem), and explain meaningful AI help or state that you did not use AI.
4. **Your test results and two limitations:** record one ordinary-use test and one invalid-input or failure test, including what you did and what actually happened. Then describe two things that could still break the program or limit what it can do. These can be normal limitations or bugs you already fixed; you do not need to leave bugs in your work.

The journal and test results can be one document. Keep them in Canvas, not in your public Git repository. Screenshots are optional; clear written results are enough.

Include in the Git README

The `README.md` should tell someone:

- What the project is for.
- Which version of Python it uses and that it needs only the standard library (`sqlite3` and `tkinter`).
- Which files to run and how to run them.
- What the project lets someone do, with a short example.
- What skills you practiced and where they appear in your code.
- Two known limitations.
- Credit for outside code/data or meaningful AI help.

The activity's top-level README gives setup information to start from. Update your own project README to describe what **you** completed; do not claim unfinished features.

Do **not** submit or publish `roster.db`, instructor solutions, textbook PDFs, private records, classmates' work, or a virtual environment. `roster.db` contains your local test data and is created automatically when the program runs.

How points are allocated

The fixed lab rubric totals 100 points. The core programming tasks are worth 60 points:

Session	Required task	Points
Week 8	Create the table and initialize the database	7

Session	Required task	Points
Week 8	Add a character with a parameterized `INSERT`	9
Week 8	List characters with `SELECT`	7
Week 8	Delete by ID with a parameterized `DELETE` and accurate result	6
Week 8	Working console menu, input checks, and persistent changes	6
Week 9	Labeled entry fields and usable grid layout	5
Week 9	Add callback with validation, database call, clearing, and refresh	8
Week 9	Refresh the roster from database results	5
Week 9	Delete callback, including no-selection feedback, and refresh	7
	Core programming tasks	**60**

The other points are 20 for Git and matching source delivery, 10 for takeaways and skills, and 10 for testing and limitations. Optional stretch goals do not replace the required tasks or add points unless the Canvas assignment says otherwise.

Stretch goals (optional)

Only try these after the required tasks work:

- Add an update function and an "Update HP" button.
- Add a search box to filter by name or class.
- Color-code rows by character class.

Vocabulary recap

Word	In everyday language
Database / SQLite	Organized information saved in one local file
Table / row / column	The roster is a table; a row is one character; a column is one detail
SQL commands	Python sends these to the database: `CREATE TABLE` sets up a table; `INSERT` adds a row; `SELECT` gets rows; `DELETE` removes a row
Safe query (`?`)	Keeps values supplied by a person separate from SQL instructions
SQL injection	A security problem caused by mixing user text into SQL instructions
Widget / grid	A widget is a window part; `.grid(...)` places parts in rows and columns
Callback (`command=...`)	The function that runs after a button is clicked